CIFAR-10 Mini-Project: Clean Classification, Adversarial Robustness, and a Convolutional VAE

STAT 695 | Name: _____

October 17, 2026

1 Experimental setup

I used the CIFAR-10 benchmark [1], which contains 50,000 training and 10,000 test RGB images in ten classes, each of size 32×32. All images were converted to tensors in $[0, 1]$; no data augmentation or channel normalization was applied. Unless noted otherwise, training used mini-batches of 128 images for 20 epochs. Pseudorandom-number generators were seeded (seed 42 for the clean experiments and seed 2026 for the attack, robust, and VAE experiments). The experiments ran with PyTorch on an NVIDIA GeForce RTX 3060 Laptop GPU.

The classifier was a 620,362-parameter CNN: three blocks of 3×3 convolution, ReLU, and 2×2 max pooling with 32, 64, and 128 output channels, respectively. The resulting $128 \times 4 \times 4$ feature map was flattened and passed through a 256-unit ReLU layer, dropout ($p = 0.3$), and a ten-logit output layer. Cross-entropy was used throughout for classification.

2 Goal 1: Clean CNN training

I trained the same CNN with two optimizers. SGD used learning rate 0.01, momentum 0.9, and weight decay 5×10^{-4}; Adam used learning rate 10^{-3} and weight decay 10^{-4}. Each run was initialized afresh with the same seed, so the comparison isolates the optimizer choice. Figure 1 reports the requested training-loss, test-loss, and test-accuracy trajectories.

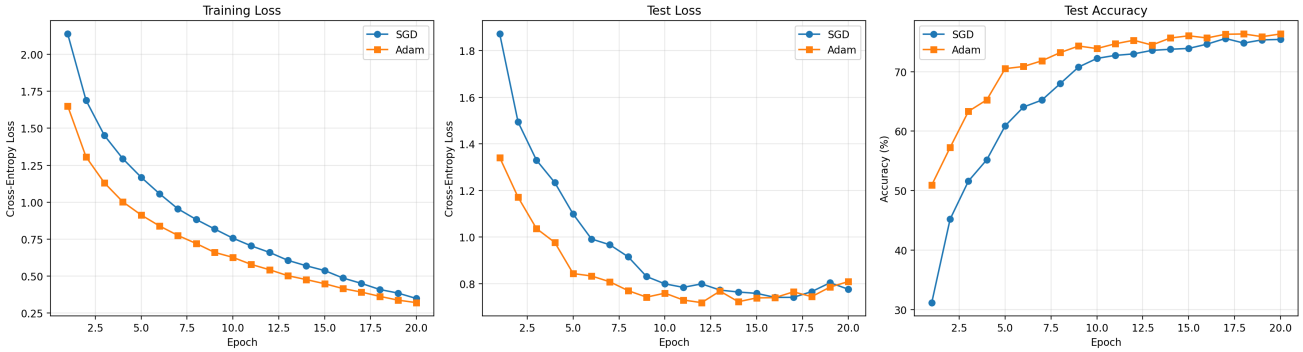


Figure 1: Clean-training trajectories over 20 epochs. Adam reduces loss and gains accuracy faster early in training; both methods plateau near the mid-70% range.

Table 1: Final-epoch clean-test results. The minimum test loss occurred before the final epoch for both runs, so final and best-epoch performance should not be conflated.

Optimizer	Final train loss	Final test loss	Final accuracy	Best accuracy	Min. test loss
SGD	0.3476	0.7758	75.42%	75.58% (epoch 17)	0.7411 (epoch 16)
Adam	0.3198	0.8090	76.34%	76.34% (epochs 18, 20)	0.7186 (epoch 12)

Adam was the preferred clean model because it reached 70.51% test accuracy by epoch 5 (versus 60.89% for SGD) and obtained the slightly higher final accuracy. Both curves show a widening train–test loss gap late in training, especially Adam after epoch 12. Thus, for model selection based only on test loss, an earlier Adam checkpoint would be preferable; the adversarial experiments below use the saved final Adam checkpoint, as implemented in the supplied attack notebook.

3 Goal 2: Attacking the clean model

I randomly selected 50 distinct test-set indices using NumPy's seeded generator (seed 2026) and attacked the final Adam CNN with the untargeted fast gradient sign method (FGSM) [2]. For an image x and label y, the attack was

$$x_{\text{adv}} = \text{clip}_{[0,1]}\left(x + \epsilon \ \text{sign}\left(\nabla_x \ell(f(x), y)\right)\right), \qquad \epsilon = \frac{8}{255} = 0.03137.$$

The measured maximum ℓ_∞ perturbation was exactly 0.031373, so the requested budget was respected. An attack was counted as successful only when an image was initially classified correctly and was then misclassified.

On the 50 selected images, the clean CNN was correct on 38 (76.00%). FGSM reduced its accuracy to 4/50 (8.00%). It successfully changed 34 of the 38 initially correct predictions, giving a conditional attack success rate of 89.47% (or 68.00% of all selected images). The 92.00% adversarial misclassification rate includes the 12 images already misclassified before the attack; the conditional success rate is therefore the more informative attack measure. As illustrated in Figure 2, the changes are hard to perceive at normal scale, while the model's label often changes (for example, ship to automobile and truck to dog).

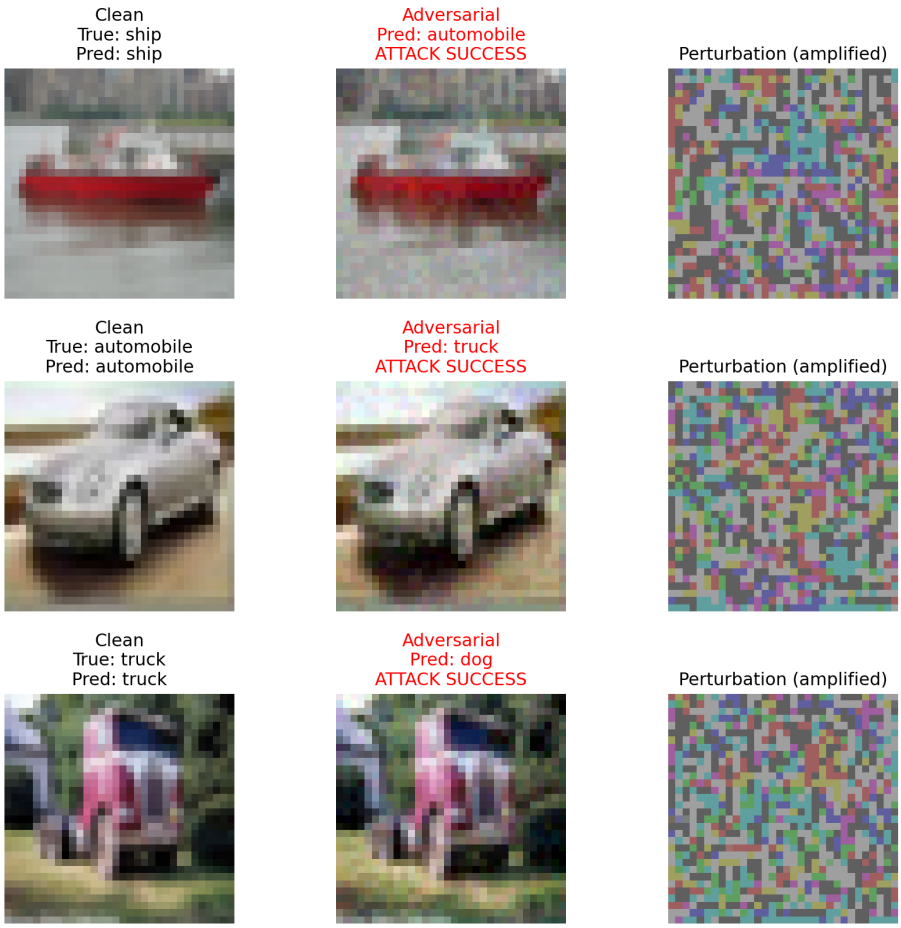


Figure 2: Representative clean/FGSM/perturbation triplets for the first three displayed test images. Perturbations are amplified only in the right column for visibility; the actual attack uses $\epsilon = 8/255$. Red labels mark successful attacks.

4 Goal 3: Adversarially robust CNN

I trained a fresh instance of the same CNN with Adam (learning rate 10^{-3}) on adversarial examples. Each training batch was perturbed with an untargeted, randomly initialized three-step projected-gradient-descent (PGD) attack: $\epsilon = 8/255$, step size $2/255$, followed by projection into the ℓ_∞ ball and clipping to $[0, 1]$. The parameter update minimized cross-entropy on these adversarial images. This is a small-scale form of the robust-optimization approach underlying PGD adversarial training [3].



Figure 3: Robust-training trajectory. The left panel is the adversarial training loss requested in Goal 3; the other panels provide clean-test context.

The adversarial training loss decreased steadily from 2.0115 to 1.3681 (Figure 3). Its final clean-test loss and accuracy were 1.0306 and 66.35%, respectively. Relative to the final clean Adam model, clean accuracy decreased by about 10 percentage points, which is the expected robustness–clean-accuracy trade-off in this setup.

I then repeated the same 8/255 FGSM evaluation on exactly the same 50 test images. Table 2 summarizes the result. Adversarial training reduced the conditional FGSM success rate from 89.47% to 37.04% and increased adversarial accuracy on this subset from 8% to 34%. The robust model nevertheless remains vulnerable: 10 initially correct images were still changed to wrong labels. Moreover, this is a 50-image, single-step-FGSM assessment, not a complete robustness certificate; stronger or more diverse attacks would be needed for a comprehensive robustness evaluation.

Table 2: FGSM evaluation at $\epsilon = 8/255$ on the identical 50 test images.

Metric	Clean Adam CNN	PGD-trained CNN
Clean-correct images	38/50 (76%)	27/50 (54%)
Adversarially correct images	4/50 (8%)	17/50 (34%)
Successful attacks	34	10
Conditional attack success rate	89.47%	37.04%
All-image attack success rate	68.00%	20.00%
Maximum observed ℓ_∞ perturbation	0.031373	0.031373

5 Goal 4: Convolutional variational autoencoder

The VAE followed the reparameterized latent-variable construction of Kingma and Welling [4]. Its encoder used three 4×4, stride-2 convolutional layers with 32, 64, and 128 channels, producing a $128 \times 4 \times 4$ representation. Two linear heads mapped it to the mean and log-variance of a 128-dimensional Gaussian latent vector. The decoder mapped a latent vector back to a feature map and used three stride-2 transposed convolutions (128 to 64 to 32 to 3 channels) with a final sigmoid. The model had 1,119,811 trainable parameters.

The VAE was trained with Adam at learning rate 10^{-3} using the sum of the per-image binary cross-entropy reconstruction loss and the KL divergence from the approximate posterior to $\mathcal{N}(0, I)$. Figure 4 gives training and test trajectories. The final training/test total losses were 1817.96/1821.96 per image,

with test reconstruction loss 1776.71 and test KL loss 45.25. The final test reconstruction BCE was 0.5784 per pixel.

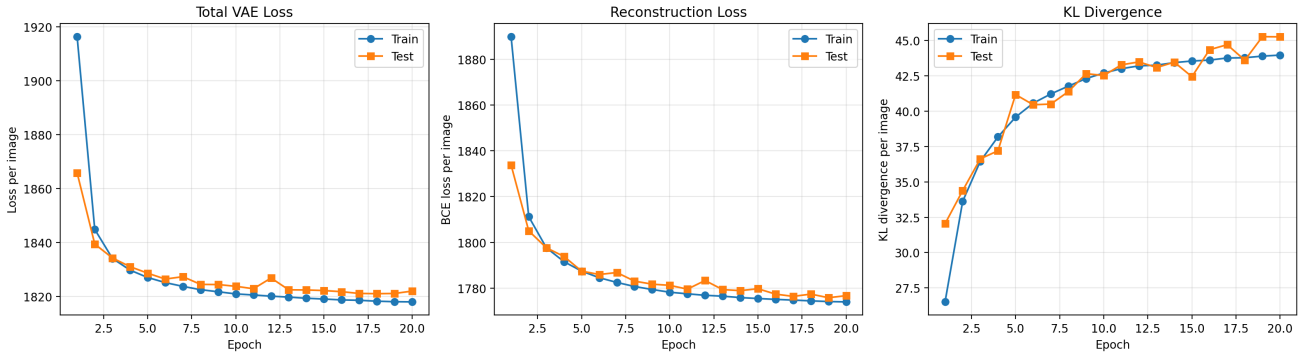


Figure 4: VAE loss components. Reconstruction error fell quickly and then plateaued, while the KL term increased as the latent representation became more active.

Figure 5 shows 64 samples decoded from standard normal latent draws. They have CIFAR-like color palettes and coarse object/background layouts, with a few vaguely vehicle- or animal-like forms. However, class identity, edges, and fine texture are generally not well resolved. This is consistent with the reconstruction examples in Figure 6: the VAE preserves broad color and rough spatial structure but produces visibly blurred reconstructions. Thus, the model learns a useful low-frequency generative representation, but its sample quality is limited by the simple architecture, short training schedule, and the usual blur-inducing behavior of pixelwise BCE VAE objectives.

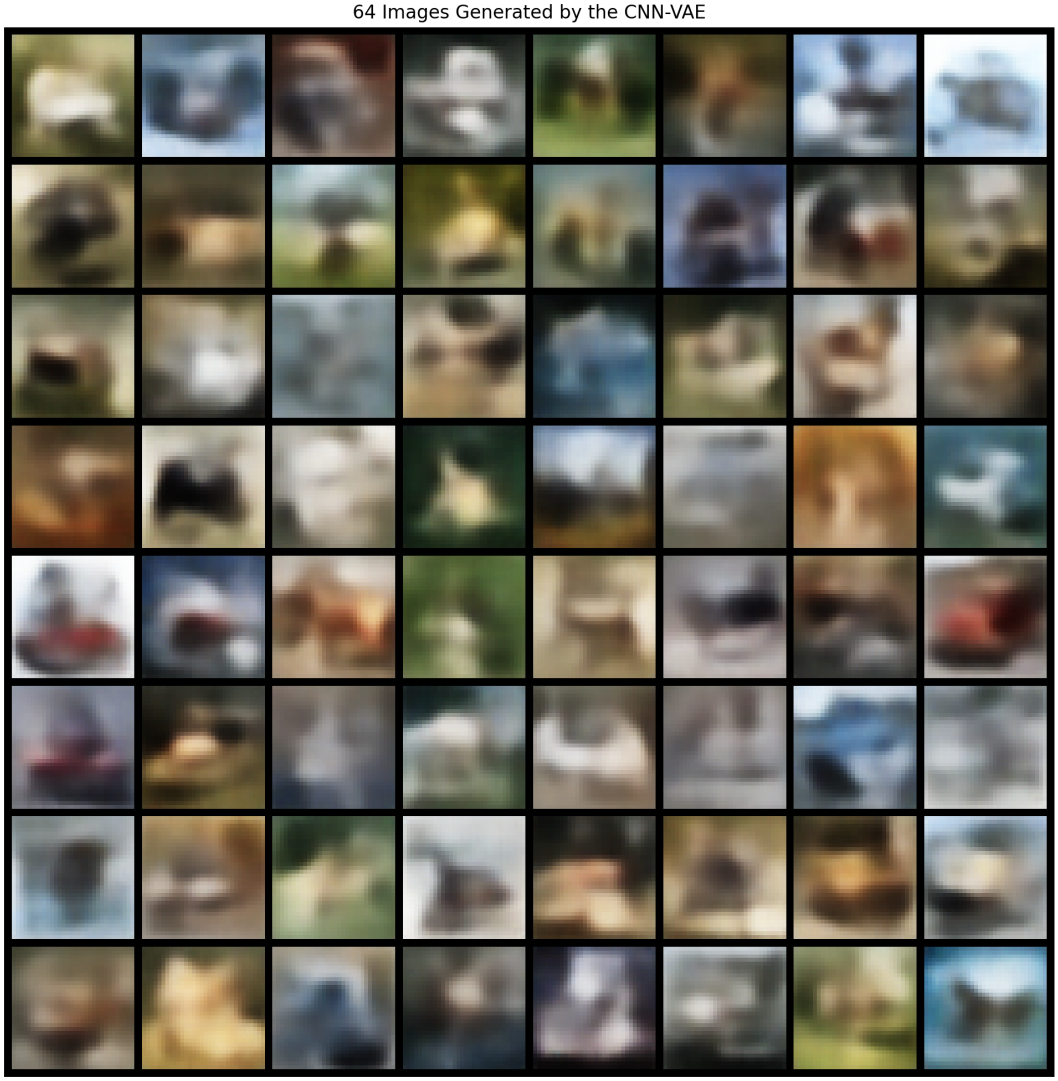


Figure 5: Sixty-four images generated by decoding samples from $\mathcal{N}(0, I)$.

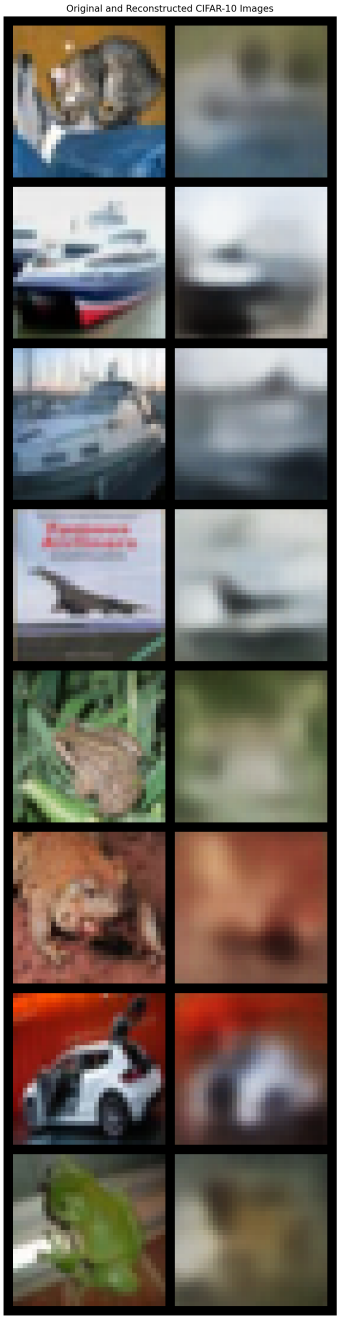


Figure 6: Eight input/reconstruction pairs (original image at left, reconstruction at right in each row). The reconstructions retain coarse scene information but are smooth and lack detail.

6 Conclusion

The clean CNN achieved 76.34% final test accuracy with Adam but was highly susceptible to ℓ_∞ FGSM perturbations of size 8/255: 89.47% of initially correct predictions were flipped on the sampled images. Three-step PGD adversarial training sacrificed clean accuracy (66.35% final clean-test accuracy) but substantially improved the matched-subset FGSM result, lowering conditional success to 37.04%. The convolutional VAE learned stable reconstructions and recognizable low-frequency samples, though generated images remained blurry. Together, the experiments demonstrate the practical tension among

clean predictive accuracy, adversarial robustness, and generative fidelity on CIFAR-10.

References

[1] A. Krizhevsky. *Learning Multiple Layers of Features from Tiny Images*. Technical report, University of Toronto, 2009. https://www.cs.toronto.edu/~kriz/learning-features-2009-TR.pdf

[2] I. J. Goodfellow, J. Shlens, and C. Szegedy. Explaining and harnessing adversarial examples. In *International Conference on Learning Representations*, 2015. https://arxiv.org/abs/1412.6572

[3] A. Madry, A. Makelov, L. Schmidt, D. Tsipras, and A. Vladu. Towards deep learning models resistant to adversarial attacks. In *International Conference on Learning Representations*, 2018. https://arxiv.org/abs/1706.06083

[4] D. P. Kingma and M. Welling. Auto-encoding variational Bayes. In *International Conference on Learning Representations*, 2014. https://arxiv.org/abs/1312.6114